NP완전 문제

NP-COMPLETENESS

네 가지 공개 코딩 문제로 배우는 알고리즘

공개 학습 문제는 네 개입니다

6~11장, 챕터별 4문제씩 총 24문제 중 기말에 10문제 출제

번호	문제	난도
9-1	제시된 부분집합 합 확인	아주 쉬움
9-2	제시된 정점 커버 확인	쉬움
9-3	제시된 해밀턴 순회 확인	기본
9-4	작은 3-CNF 식의 만족 가능성	조금 도전

기말은 별도 객관식·서술형 없이 코드 실행 결과로 평가합니다.

답을 찾기와 답을 확인하기

같은 문제도 두 작업의 계산량은 다를 수 있습니다.

찾기

조건을 만족하는 선택이나 경로를 직접 구합니다.

확인하기

누군가 제시한 선택이나 경로가 맞는지 검사합니다.

이번 실습

앞의 세 문제는 확인, 마지막은 작은 입력에서 찾기입니다.

P와 NP를 정확하게 읽습니다

여기서 다루는 복잡도 분류는 예/아니요로 답하는 결정 문제 기준입니다.

P

입력 크기의 다항 시간 안에 풀 수 있는 결정 문제입니다.

NP

예라는 답의 증거를 다항 시간 안에 확인할 수 있습니다.

NP완전

NP에 속하며, NP의 모든 문제를 이 문제로 다항 시간에 변환할 수 있습니다.

작은 입력은 전부 살펴볼 수 있습니다

3-CNF 실습의 변수는 최대 12개로 제한합니다.

값 배정

변수가 n개이면 0/1 배정은 2^n개입니다.

변수 12개

4,096가지 배정을 차례대로 확인할 수 있습니다.

변수 30개

1,073,741,824가지로 늘어나 같은 방법이 부담됩니다.

약 2시간 수업 흐름

한 문제를 이해하고 직접 구현하는 시간을 함께 확보합니다.

시간	내용
0–20분	부분집합 합의 증거 확인
20–45분	정점 커버의 증거 확인
45–55분	쉬는 시간
55–80분	해밀턴 순회 검사
80–105분	작은 3-CNF 완전 탐색
105–120분	직접 구현·경계 테스트

마지막에는 빈 코드에서 다시 작성하고 테스트합니다.

9-1 제시된 부분집합 합 확인

아주 쉬움

정수 배열 a와 같은 길이의 0/1 배열 selected가 주어집니다. selected[i]=1인 원소만 고른 합이 target과 같은지 확인하세요. 길이는 0..20, a의 값은 0..100, target은 0..2000입니다. 같으면 1, 다르면 0을 반환합니다. 새 부분집합을 찾는 문제가 아니라 이미 제시된 선택을 검사합니다.

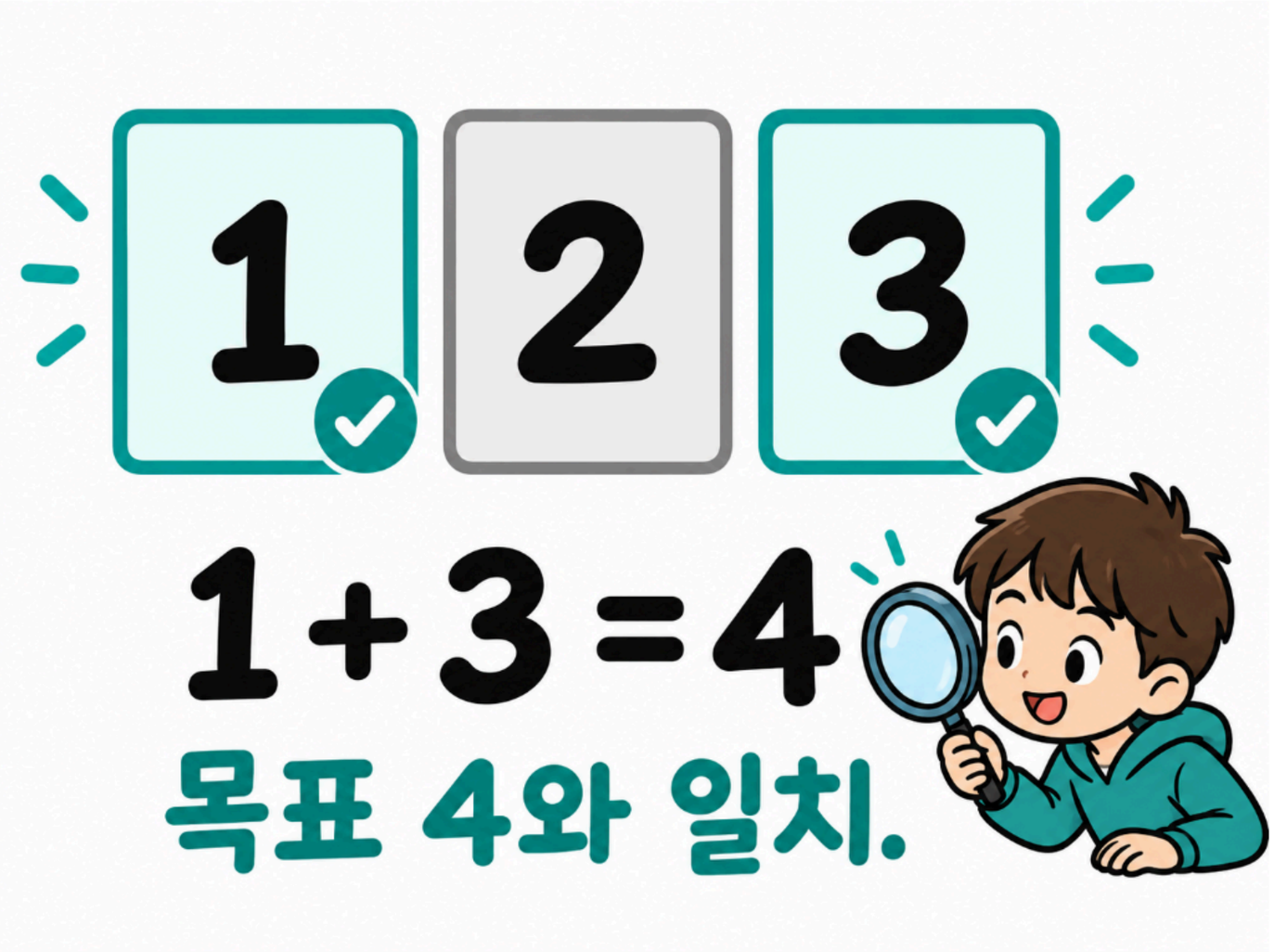
함수와 공개 예제

Python: verify_subset_sum(a, selected, target)

입력	반환값
[1,2,3], [1,0,1], 4	1
[1,2,3], [1,0,1], 3	0
[], [], 0	1

int verify_subset_sum(const int a[], const int selected[], int n, int target)

제시된 부분집합 합 확인의 작은 예



선택 표시 1,0,1
1과 3을 골랐습니다.
합 4는 목표 4와
같습니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

원소	선택 표시	합에 더한 값
1	1	1
2	0	0
3	1	3
합계		4

선택 표시가 1인 칸만 더하고 목표와 같은지 한 번 비교합니다. 가능한 모든 부분집합을 탐색할 필요가 없습니다.

이 문제의 계산 방법

9-1

선택 표시가 1인 칸만 더하고 목표와 같은지 한 번 비교합니다. 가능한 모든 부분집합을 탐색할 필요가 없습니다.

아무것도 고르지 않으면 합은 0입니다. a와 selected의 길이는 같고 선택 표시는 0 또는 1로 주어집니다.

제시된 부분집합 합 확인 · Python

```
def verify_subset_sum(a, selected, target):
    total = 0
    for i in range(len(a)):
        if selected[i]:
            total += a[i]
    return int(total == target)
```

시간 O(n), 추가 공간 O(1)

제시된 부분집합 합 확인 · C

```
int verify_subset_sum(const int a[], const int selected[],
                      int n, int target) {
    int total=0;
    for (int i=0; i<n; ++i) if (selected[i]) total+=a[i];
    return total==target;
}
```

시간 O(n), 추가 공간 O(1)

경계 입력과 복잡도

9-1

경계 입력

아무것도 고르지 않으면 합은 0입니다. a와 selected의 길이는 같고 선택 표시는 0 또는 1로 주어집니다.

계산량

시간 O(n), 추가 공간 O(1)

직접 작성하고 수정합니다

9-1 제시된 부분집합 합 확인

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

9-2 제시된 정점 커버 확인

쉬움

정점 1..8개의 무방향 그래프가 대칭인 0/1 인접 행렬 graph로 주어지며 대각선은 0입니다. selected[i]는 정점 i를 골랐으면 1, 아니면 0입니다. 모든 간선의 양 끝 중 적어도 하나가 선택되었는지 검사하세요. 모두 덮었으면 1, 덮이지 않은 간선이 하나라도 있으면 0을 반환합니다. 선택 개수를 최소화할 필요는 없습니다.

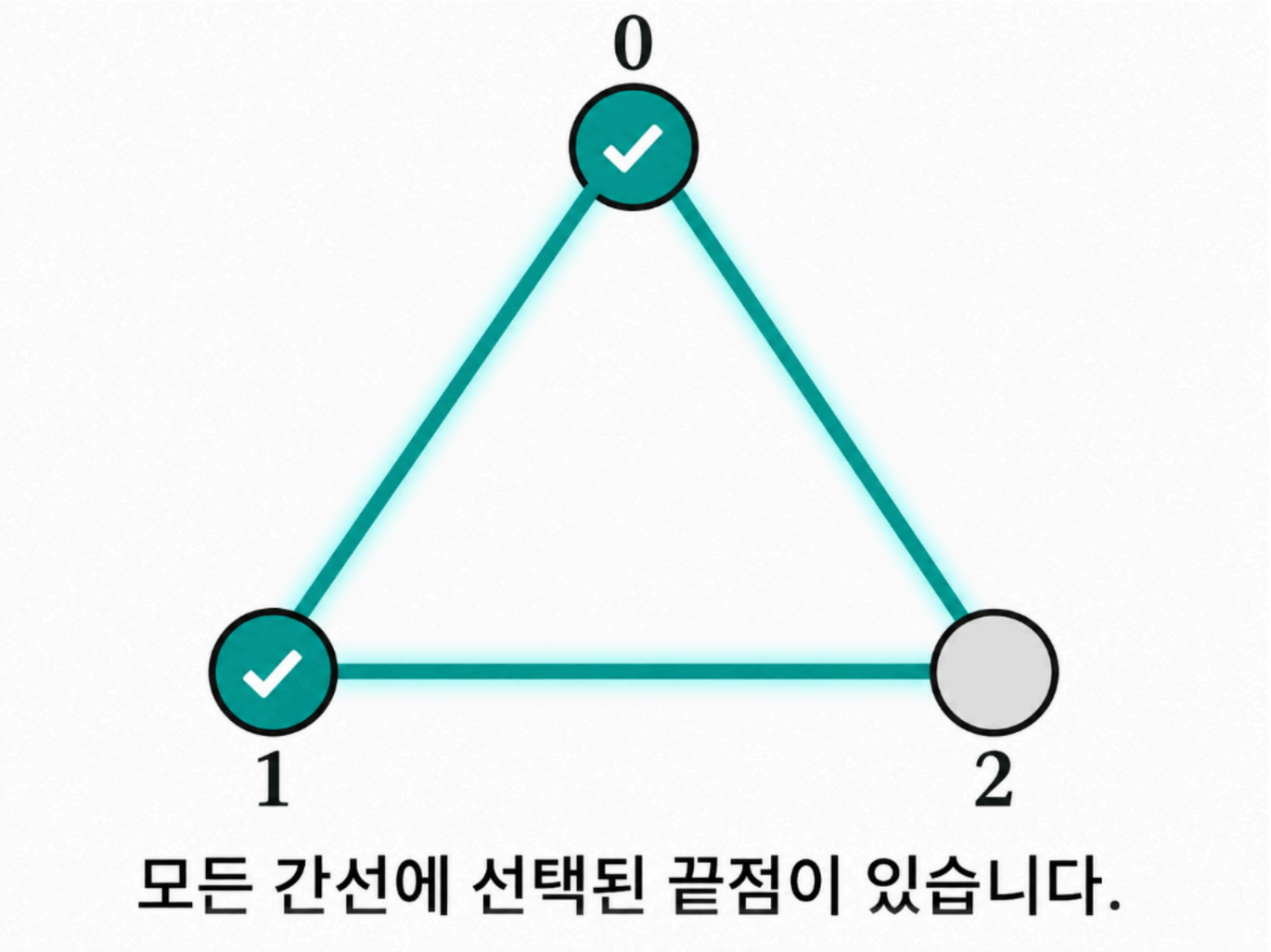
함수와 공개 예제

Python: verify_vertex_cover(graph, selected)

입력	반환값
[[0,1,1],[1,0,1],[1,1,0]], [1,1,0]	1
[[0,1,1],[1,0,1],[1,1,0]], [1,0,0]	0
[[0]], [0]	1

int verify_vertex_cover(const int graph[][8], int n, const int selected[])

제시된 정점 커버 확인의 작은 예



정점 0과 1을
선택했습니다.
삼각형의 모든 간선에
선택한 끝점이 있습니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

간선	선택한 끝점	덮었는가
0–1	0, 1	예
0–2	0	예
1–2	1	예

존재하는 간선을 하나씩 봅니다. 양 끝이 모두 미선택인 간선 하나를 찾으면 실패입니다.

이 문제의 계산 방법

9-2

존재하는 간선을 하나씩 봅니다. 양 끝이 모두 미선택인 간선 하나를 찾으면 실패입니다.

간선이 없으면 아무 정점도 고르지 않아도 모든 간선을 덮은 것으로 처리합니다. 양 끝을 모두 고르는 것도 허용합니다.

제시된 정점 커버 확인 · Python

```
def verify_vertex_cover(graph, selected):
    n = len(graph)
    for i in range(n):
        for j in range(i+1, n):
            if graph[i][j] and not selected[i] and not selected[j]:
                return 0
    return 1
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

제시된 정점 커버 확인 · C

```
int verify_vertex_cover(const int graph[][8], int n,
                        const int selected[]) {
    for (int i=0; i<n; ++i)
        for (int j=i+1; j<n; ++j)
            if (graph[i][j] && !selected[i] && !selected[j])
                return 0;
    return 1;
}
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

경계 입력과 복잡도

9-2

경계 입력

간선이 없으면 아무 정점도 고르지 않아도 모든 간선을 덮은 것으로 처리합니다. 양 끝을 모두 고르는 것도 허용합니다.

계산량

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

직접 작성하고 수정합니다

9-2 제시된 정점 커버 확인

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

10분 쉬었다가 이어갑니다

앞의 두 계산을 정리합니다.

뒤의 문제는 앞에서 익힌 계산을
반복하거나 탐색에 연결합니다.

9-3 제시된 해밀턴 순회 확인

기본

정점 n개(n=3..8)의 대칭 0/1 인접 행렬 graph와 길이 n인 배열 path가 주어집니다. path가 0..n–1을 한 번씩 포함하고, 이 순서의 모든 이웃 정점 사이와 마지막에서 처음으로 가는 간선이 존재하면 1을 반환하세요. 중복·범위 밖 번호·빠진 연결이 있으면 0입니다. 경로를 새로 찾을 필요는 없습니다.

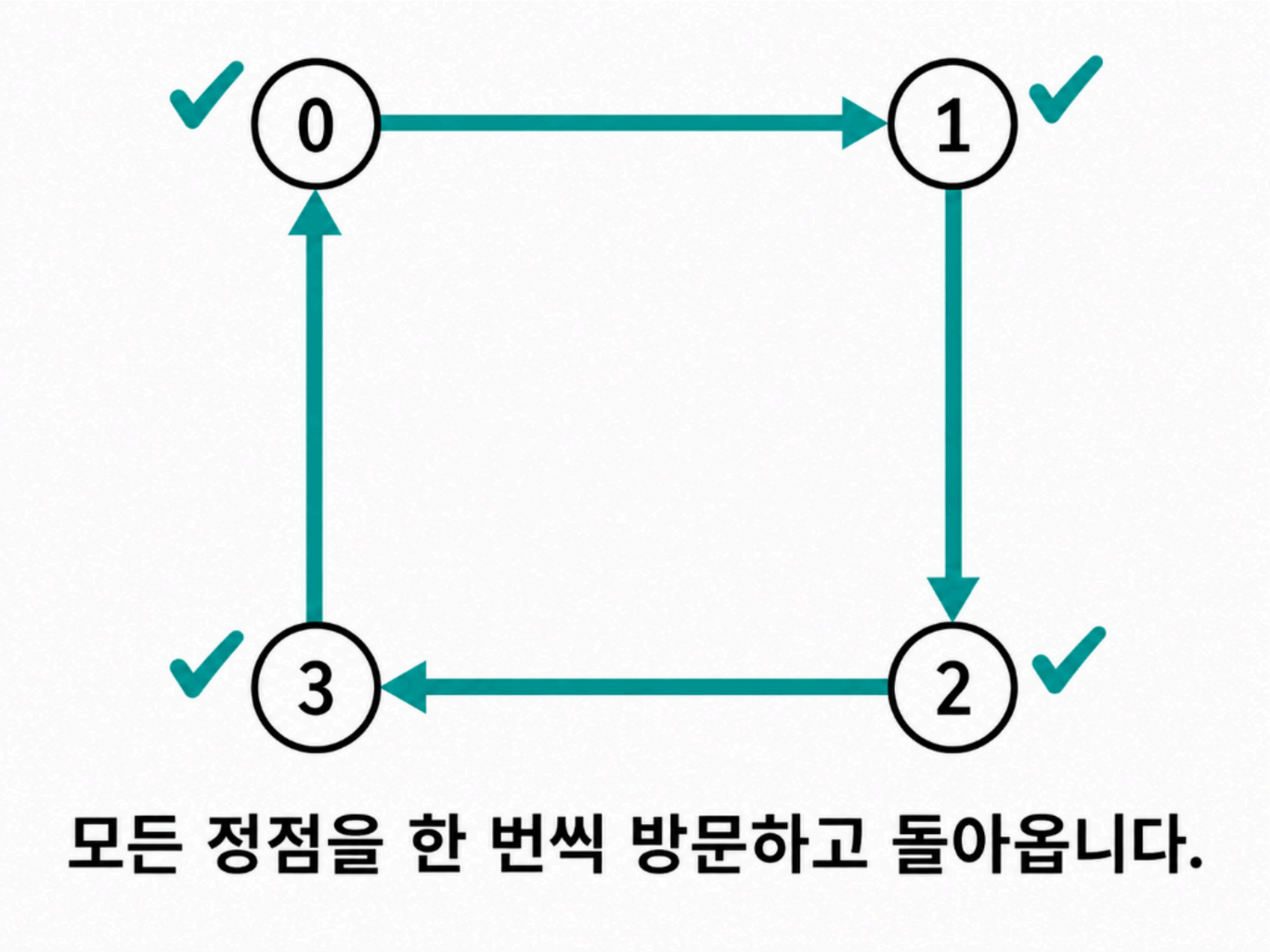
함수와 공개 예제

Python: verify_hamiltonian(graph, path)

입력	반환값
[[0,1,1],[1,0,1],[1,1,0]], [0,1,2]	1
[[0,1,1],[1,0,1],[1,1,0]], [0,1,1]	0
[[0,1,0],[1,0,1],[0,1,0]], [0,1,2]	0

int verify_hamiltonian(const int graph[][8], int n, const int path[])

제시된 해밀턴 순회 확인의 작은 예



0,1,2,3,0
모든 정점을 한 번씩 지나고 출발점으로 돌아옵니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

확인 내용	결과
0,1,2,3이 한 번씩 등장	통과
0–1, 1–2, 2–3 간선	존재
마지막 3–0 간선	존재
반환값	1

먼저 정점 번호의 범위와 중복을 확인합니다. 그다음 순서대로 이어지는 간선과 출발점으로 돌아오는 간선을 확인합니다.

이 문제의 계산 방법

9-3

먼저 정점 번호의 범위와 중복을 확인합니다. 그다음 순서대로 이어지는 간선과 출발점으로 돌아오는 간선을 확인합니다.

범위 밖 정점 번호로 행렬을 읽기 전에 번호를 먼저 검사합니다. 마지막에서 첫 정점으로 돌아오는 간선도 필요합니다.

제시된 해밀턴 순회 확인 · Python

```
def verify_hamiltonian(graph, path):
    n = len(graph)
    seen = [False]*n
    for v in path:
        if v < 0 or v >= n or seen[v]:
            return 0
        seen[v] = True
    for i in range(n):
        if not graph[path[i]][path[(i+1)%n]]:
            return 0
    return 1
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

제시된 해밀턴 순회 확인 · C

```
int verify_hamiltonian(const int graph[][8], int n,
                       const int path[]) {
    int seen[8]={0};
    for (int i=0; i<n; ++i) {
        int v=path[i];
        if (v<0 || v>=n || seen[v]) return 0;
        seen[v]=1;
    }
    for (int i=0; i<n; ++i)
        if (!graph[path[i]][path[(i+1)%n]]) return 0;
    return 1;
}
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

경계 입력과 복잡도

9-3

경계 입력

범위 밖 정점 번호로 행렬을 읽기 전에 번호를 먼저 검사합니다. 마지막에서 첫 정점으로 돌아오는 간선도 필요합니다.

계산량

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

직접 작성하고 수정합니다

9-3 제시된 해밀턴 순회 확인

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

9-4 작은 3-CNF 식의 만족 가능성

조금 도전

불 변수 n개(1..12)와 절 0..30개가 주어집니다. 각 절은 리터럴 정수 3개입니다. 양수 k는 x_k, 음수 –k는 NOT x_k를 뜻하며 1≤|k|≤n입니다. 한 절 안의 세 리터럴은 OR, 모든 절은 AND로 묶습니다. 모든 절을 참으로 만드는 0/1 값 배정이 있으면 1, 없으면 0을 반환하세요. 절이 0개이면 1입니다.

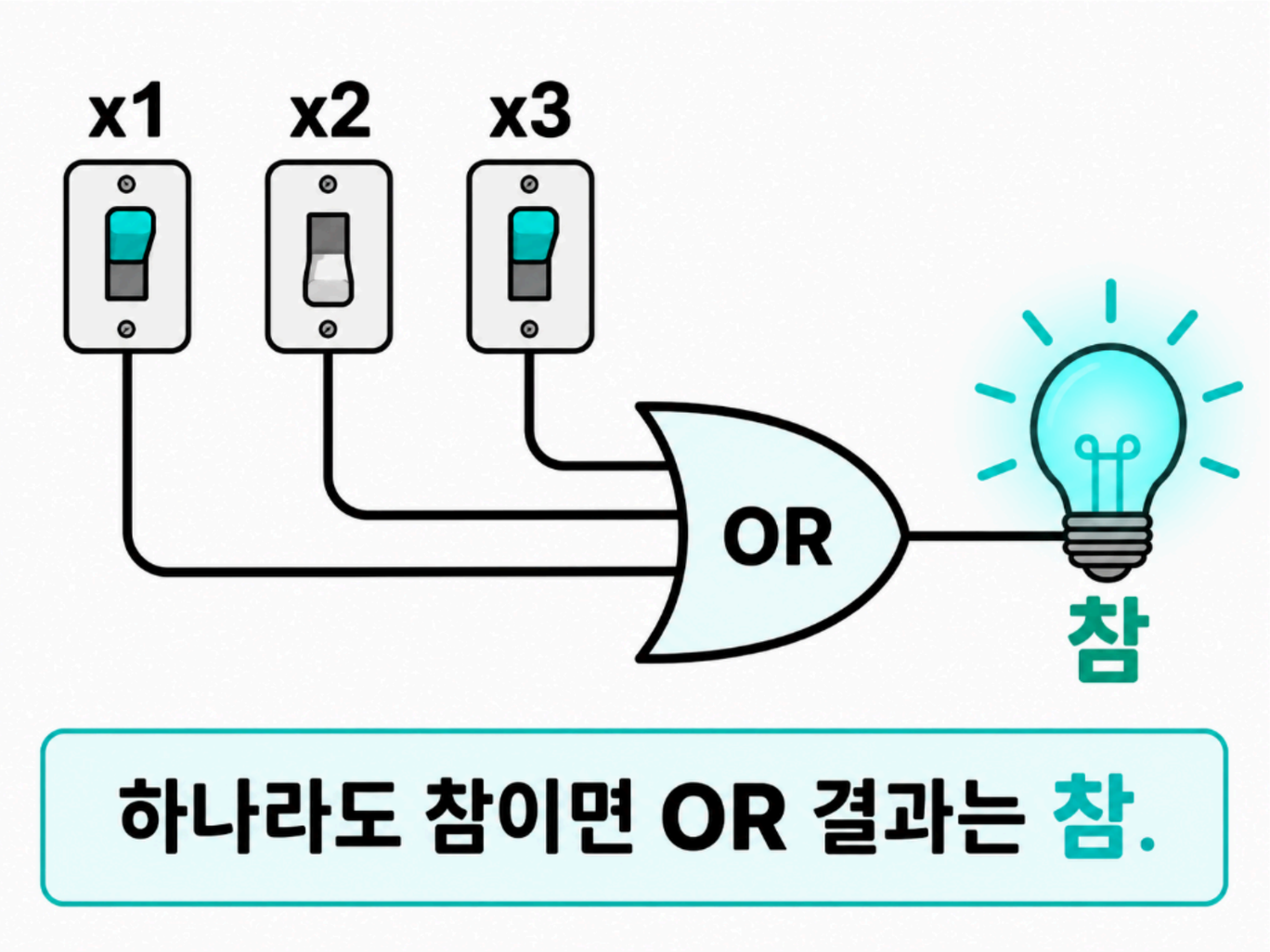
함수와 공개 예제

Python: satisfiable_3cnf(n, clauses)

입력	반환값
1, [[1,1,1]]	1
1, [[1,1,1],[-1,-1,-1]]	0
2, []	1

int satisfiable_3cnf(int n, const int clauses[][3], int m)

작은 3-CNF 식의 만족 가능성의 작은 예



$x_1=1, x_2=0, x_3=1$
이 절은 OR입니다.
하나라도 참이면
절 전체가 참입니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

절	$x_1=0$	$x_1=1$
(x_1 OR x_1 OR x_1)	0	1
(NOT x_1 OR NOT x_1 OR NOT x_1)	1	0
두 절을 모두 만족?	아니요	아니요
만족 가능한 배정	없음 → 0	

n개의 0/1 값 배정은 2^n개입니다. 각 배정에서 절마다 리터럴 셋 중 하나라도 참인지 확인합니다.

이 문제의 계산 방법

9-4

n개의 0/1 값 배정은 2^n개입니다. 각 배정에서 절마다 리터럴 셋 중 하나라도 참인지 확인합니다.

음수 리터럴은 변수 값을 반대로 읽습니다. 한 절이라도 거짓이면 그 배정은 실패이며, 절이 없으면 모든 배정이 조건을 만족합니다.

0/1 값 배정을 정수 하나로 저장합니다

오른쪽 끝 비트가 x_1, 그 왼쪽 비트가 x_2입니다.

정수 mask	두 비트	x_1	x_2
0	00	0	0
1	01	1	0
2	10	0	1
3	11	1	1

0부터 2^n-1까지 반복하면 모든 0/1 배정을 한 번씩 봅니다.

리터럴은 변수 또는 반대 값입니다

k번 변수의 값은 (mask >> (k–1)) & 1로 읽습니다.

>> (k–1)

읽을 비트를 오른쪽 끝으로 옮깁니다. & 1은 그 한 자리만 남깁니다.

리터럴 k

k번 변수의 0/1 값을 그대로 사용합니다.

리터럴 –k

값이 0이면 1, 1이면 0으로 바꿉니다. 코드는 1–value입니다.

작은 3-CNF 식의 만족 가능성 · Python

```
def satisfiable_3cnf(n, clauses):
    for mask in range(1 << n):
        satisfied = True
        for clause in clauses:
            clause_ok = False
            for literal in clause:
                value = (mask >> (abs(literal)-1)) & 1
                if literal < 0: value = 1-value
                if value: clause_ok = True
            if not clause_ok:
                satisfied = False
                break
        if satisfied: return 1
    return 0
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

작은 3-CNF 식의 만족 가능성 · C

```
int satisfiable_3cnf(int n, const int clauses[][3], int m) {
    for (int mask=0; mask<(1<<n); ++mask) {
        int ok=1;
        for (int i=0; i<m; ++i) {
            int clause_ok=0;
            for (int j=0; j<3; ++j) {
                int lit=clauses[i][j], k=lit<0?-lit:lit;
                int value=(mask>>(k-1))&1;
                if (lit<0) value=1-value;
                if (value) clause_ok=1;
            }
            if (!clause_ok) {ok=0; break;}
        }
        if (ok) return 1;
    } return 0;
}
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

경계 입력과 복잡도

9-4

경계 입력

음수 리터럴은 변수 값을 반대로 읽습니다. 한 절이라도 거짓이면 그 배정은 실패이며, 절이 없으면 모든 배정이 조건을 만족합니다.

계산량

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

직접 작성하고 수정합니다

9-4 작은 3-CNF 식의 만족 가능성

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

이 장의 네 문제 정리

강의노트와 사이트는 같은 함수 규격을 사용합니다.

문제	함수	난도
9-1	verify_subset_sum	아주 쉬움
9-2	verify_vertex_cover	쉬움
9-3	verify_hamiltonian	기본
9-4	satisfiable_3cnf	조금 도전

공개 문제의 조건 안에서 올바른 반환값을 구합니다.

기말 준비

공개 24문제 중 10문제 출제

6~11장, 챕터마다 네 문제를 공부합니다.
정해진 함수 인자와 반환값을 지킵니다.
직접 작성한 코드의 실행 결과로 평가합니다.

NP완전 문제 정리

NP-COMPLETENESS

9-1 제시된 부분집합 합 확인
9-2 제시된 정점 커버 확인
9-3 제시된 해밀턴 순회 확인
9-4 작은 3-CNF 식의 만족 가능성